# VeriLM: A Commit-and-Audit Protocol for Open-Weight LLM Inference

Federico Carrone, Diego Kingston, Mariano Nicolini, Pedro Fontana, Manuel Puebla
LambdaClass

*Preprint*

**Abstract —** Users of open-weight LLMs have no technical mechanism to verify which model actually ran, which decode policy was used, or whether the output was altered after generation. We present VeriLM, a commit-and-audit protocol for open-weight LLM inference. The core insight is that public weights permit lightweight audit rather than expensive proof systems. VeriLM deploys as a sidecar over unmodified serving stacks: each response carries a compact receipt, and only audited responses open traces. The final protocol provides four guarantee classes: exact checks for preprocessing, model identity, shell matmuls, bridge operations, the final-token tail, and decode/output replay; statistical prefix-state provenance in routine audit mode; approximate attention replay bounded by the requantization corridor; and explicit fail-closed rejection of unsupported semantics. Because receipts are small and audits are rare, VeriLM supports continuous random auditing at low amortized cost.

## 1 Introduction

Open-weight LLM inference presents a trust problem: the client sends a prompt to a provider who claims to run a specific model (e.g., Llama 70B), but the client has no way to verify that the provider actually used those weights. The economic incentive to cheat is clear — serving a smaller or more aggressively quantized model reduces compute costs while the client pays the same price.

Existing approaches to verifiable inference fall into two categories. Cryptographic proof systems and specialized verifiable-inference systems such as SafetyNets [1] and zkCNN [2] can prove arbitrary or restricted computations, but impose overhead that makes real-time LLM inference impractical. Trusted execution environments (TEEs) provide attestation [3] but require hardware trust assumptions and limit deployment flexibility.

VeriLM takes a different approach: a lightweight sidecar protocol that runs alongside unmodified GPU inference. The key insight is that INT8 quantized inference consists of operations that are either deterministic or canonically recomputable, and can be verified exactly. The only non-deterministic component is FP16/BF16 attention, which the protocol constrains from both sides without requiring exact reproduction.

The protocol provides exact, approximate, statistical, and fail-closed guarantees in different parts of the pipeline. The honest decomposition is: exact verification everywhere outside the attention interior, approximate attention replay, and statistical prefix anchoring unless deep audit is used.

## 2 System Model

### 2.1 Threat Model

The adversary controls the inference server. They may swap model weights, modify attention computation, fabricate intermediate activations, or lie about earlier context. The adversary may cheat adaptively — choosing different strategies for different responses. The following timing and knowledge constraints apply:

1. **Commitment precedes challenge.** The provider sends the receipt ($R_T$, $R_{\text{KV}}$, $M$, $H(s)$, $N$, ...) before learning whether the response will be audited.
2. **Unpredictable selection.** The provider cannot predict which response, token position, or layer the verifier will challenge.
3. **Secret key.** The verifier holds $r_j$ and $v_j^{(i)}$ secret; the provider never observes them or the audit decision rule.
4. **Local verification.** The verifier checks locally — no third party sees the challenge, the opening, or the conversation.

In the intended deployment, the client's verifier software automatically and randomly audits a small fraction of responses (e.g., 1–5%).

The client holds a verifier key ($\sim 26$ MB for Llama 70B) derived from the public model weights and secret verifier-side randomness. The client audits responses directly — no trusted third party sees the conversation. The attester / verifier / relying-party terminology here follows standard remote-attestation usage [3].

VeriLM provides interactive, client-held verification. The client who holds the verifier key checks the computation directly. The protocol does not produce a transferable proof that third parties can verify offline — this is a deliberate tradeoff for low overhead. Audit openings contain intermediate activations from which the conversation content can be reconstructed; this is why the client audits themselves rather than delegating to a third party.

### 2.2 Design Principles

VeriLM is designed around three constraints:

1. **No serving-path changes.** The provider runs unmodified GPU inference. The only addition is a tracing layer that captures intermediates alongside normal execution.
2. **Minimal per-response overhead.** Each response carries a compact receipt binding trace commitments, deployment specs, transcript randomness, and token count. Most responses are never audited.
3. **Client-side verification.** The client performs all verification using CPU-feasible operations (dot products, hash checks, and — for attention replay — matrix arithmetic scaling as $O(n^2)$ in sequence length).

**Protocol guarantees at a glance.**

1. **Exact.** Preprocessing, model identity, shell matmuls, bridge operations, the final-token tail, and decode/output replay.
2. **Statistical.** Prefix KV correctness under routine audit sampling ($k$ prefix positions checked).

3. **Approximate.** Attention replay under the FP16↔FP64 requantization corridor (Section 6).
4. **Outside protocol scope.** Standard cryptographic assumptions, verifier-secret secrecy, no side-channel leakage of verifier secrets, and correct verifier execution.

### 2.3 Protocol Boundary

The final protocol uses four guarantee classes:

1. **Exact.** A property is cryptographically checked or canonically recomputed with fully specified semantics.
2. **Approximate.** The verifier independently replays the computation and constrains it tightly, but native FP16/BF16 execution is not bit-reproducible across hardware.
3. **Statistical.** Commitment binding is exact, but correctness of unopened positions depends on challenge sampling unless deep audit is used.
4. **Fail-closed.** A feature is either replayed exactly or rejected explicitly; the verifier never silently accepts unsupported semantics.

VeriLM does *not* assume honest prover hardware or honest provider runtime behavior. Those are what the protocol is designed to remove. The explicit assumptions outside protocol scope are standard cryptographic assumptions, secrecy of verifier-only material, resistance to side-channel leakage of verifier secrets, and correct execution of the verifier itself.

The final protocol targets autoregressive decoder-only transformers with a committed capture layout and replay specification. Unsupported architectures must fail closed rather than be silently interpreted as if they shared decoder-only semantics.

| Symbol | Meaning |
|---|---|
| $R_W$ | Merkle root over model weights (public identity) |
| $R_T$ | Merkle root over trace intermediates |
| $R_{\text{KV}}$ | Merkle root over per-token $K, V$ history |
| $M$ | Deployment commitment over input/model/decode/output specs |
| $H(s)$ | Transcript-seed commitment |
| $N$ | Token count in response |
| $c, \ell$ | Number of challenged tokens; number of opened layers per token |
| $m$ | Output dimension of the checked weight matrix (length of $r_j$) |
| $k$ | Sampled prefix positions for KV provenance |
| $\tau$ | Acceptance threshold for attention replay |
| $p$ | Prime modulus for Freivalds checks ($p \geq 2^{32}$); arithmetic in $\mathbb{F}_p$ |

Table 1: Notation

## 3 Verification Layers

VeriLM's verification is structured in six layers. This section defines the taxonomy and key terms; the full procedure is specified in Section 4.

The **shell** is the non-attention path through each transformer layer: seven INT8 weight matrix multiplications ($W_q$, $W_k$, $W_v$, $W_o$, $W_{\text{gate}}$, $W_{\text{up}}$, $W_{\text{down}}$), **requantization bridges** (the i32 → i8 conversions between consecutive matmul stages), RoPE, RMSNorm, and SiLU. The shell includes nonlinear operations (RMSNorm, SiLU), but all shell operations are deterministic or canonically recomputable. Its exactness is a composition: cryptographic checks (Freivalds [4]) on the weight matmuls, plus deterministic or canonical recomputation on the bridge operations.

Attention ($QK^T$, softmax, $\alpha V$) is computed in FP16/BF16, which is not bit-reproducible across hardware — this is the only non-exact component. The final exact tail starts from a captured pre-final-norm residual after the last layer, applies final RMSNorm exactly, binds the LM head with Freivalds, computes logits exactly, and replays the decode/output policy.

The six layers are:

1. **Input/model binding (exact).** Preprocessing policy, model identity, quantization scheme, architecture parameters, and deployment specs are committed and checked against known-good values.
2. **Shell verification (exact).** Cryptographic Freivalds checks ($\leq 1/2^{32}$ false-accept) on all shell weight matmuls, plus deterministic recomputation of requantization bridges, RoPE, RMSNorm, and SiLU.
3. **KV provenance (statistical by default).** Merkle-committed per-token K,V history; sampled positions are shell-verified. Binding is exact; correctness of unsampled positions depends on sampling rate unless deep audit is used.
4. **Cross-layer consistency (structural).** Opening multiple layers on the same token creates algebraic coupling through the residual stream — fake attention must stay consistent across all opened layers.
5. **Attention replay (approximate).** The verifier recomputes attention from shell-verified Q and committed prefix K,V in FP64, quantizes to INT8, and compares. Limited by FP16↔FP64 mismatch.
6. **Final-token replay (exact / fail-closed).** The verifier starts from the captured pre-final-norm residual, checks the LM head with Freivalds, computes logits exactly, and replays the decode/output policy; unsupported semantics are rejected explicitly rather than silently accepted.

Table 2 traces the full forward pass for one output token. Each operation is labeled with its verification type: Freivalds-checked weight matmuls, canonically recomputable bridge operations, and the single non-exact point — FP16 attention. The only non-replayable provider-side state is the per-layer `attn_out_i8` and its quantization scale, plus the captured pre-final-norm residual that anchors the exact final-token tail.

```
residual = embedding[token_id]                          exact lookup
for each layer:
  x_norm = RMSNorm(residual)                            canonical
  x_i8, s = quantize(x_norm)                            exact
  q_i32 = W_q @ x_i8                                    Freivalds
  k_i32 = W_k @ x_i8                                    Freivalds
  v_i32 = W_v @ x_i8                                    Freivalds
  q = RoPE(dequant(q_i32, ...), pos)                    canonical
  k = RoPE(dequant(k_i32, ...), pos)                    canonical
  v = dequant(v_i32, ...)                               canonical
  attn = softmax(q @ k^T / sqrt(d))                     non-exact
@ v
  attn_out_i8, sa = quantize(attn)                      STORED
  o_i32 = W_o @ attn_out_i8                             Freivalds
  residual += dequant(o_i32, ...)                       canonical
  x_norm = RMSNorm(residual)                            canonical
  x_i8, s = quantize(x_norm)                            exact
  gate_i32 = W_gate @ x_i8                              Freivalds
  up_i32   = W_up   @ x_i8                              Freivalds
    x  =  SiLU(dequant(gate_i32))  *                    canonical
dequant(up_i32)
  x_i8, s = quantize(x)                                 exact
  down_i32 = W_down @ x_i8                              Freivalds
  residual += dequant(down_i32, ...)                    canonical
final_residual = residual                               STORED
x_norm = RMSNorm(final_residual)                        canonical
x_i8, s = quantize(x_norm)                              exact
logits_i32 = LM_head @ x_i8                             Freivalds + exact logits
token = canonical_decode(logits_i32,                    exact / fail-closed
seed, policy)
```

Table 2: Annotated forward pass for one output token. Each operation is labeled with its verification type. The only non-exact stage is FP16 attention; the final-token tail is exact because it starts from the captured pre-final-norm residual.

| Component | Verification method | Guarantee | What is bound |
|---|---|---|---|
| Input preprocessing | Committed input spec | Exact | Tokenizer, chat template, BOS/EOS, truncation, special-token handling, system prompt |
| Embedding | Merkle proof to committed embedding root | Exact | Token-to-row binding |
| Shell matmuls | Freivalds | Exact | Seven shell matrix families |
| Bridge operations | Canonical recomputation | Exact | Requantization, residual, RMSNorm, RoPE, SiLU |
| Prefix / KV provenance | Merkle binding + sampled shell checks or deep audit | Statistical / Exact | Committed prefix state |
| Attention | Independent replay against committed prefix/output | Approximate | Consistency with committed shell values |
| Final boundary | Captured pre-final-norm residual | Exact | Start of the exact final-token tail |
| LM head | Freivalds + exact logits replay | Exact | Final linear map and logits |
| Decode policy | Canonical replay or explicit rejection | Exact / Fail-closed | Sampler semantics and randomness |
| Output policy | Exact replay or explicit rejection | Exact / Fail-closed | Stopping and text-cleanup semantics |

Table 3: Verification coverage matrix for the final protocol. "Fail-closed" means a feature is either replayed exactly or rejected explicitly; the verifier never silently accepts unsupported semantics.

## 4 The Protocol

### 4.1 Phase 0: Setup

**Public identity.** A Merkle root [5] $R_W$ is computed over all weight matrices of the published checkpoint. This root is the model's public identity — anyone can recompute it from the published weights.

**Verifier key generation.** The verifier fixes a prime $p \geq 2^{32}$. All Freivalds checks operate in $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$: INT8 inputs and INT32 accumulators are lifted into this field. For the 7 shell matrix types ($W_q$, $W_k$, $W_v$, $W_o$, $W_{\text{gate}}$, $W_{\text{up}}$, $W_{\text{down}}$) and the final LM head, the verifier samples secret random vectors $r_j$ uniformly from $\mathbb{F}_p^m$. For each per-layer matrix, the verifier precomputes $v_j^{(i)} = r_j^T W_j^{(i)} \bmod p$; for the LM head, the verifier precomputes the corresponding verifier-side check vector once for the global unembedding matrix. The verifier key also binds the embedding commitment, final RMSNorm weights, model

configuration, quantization metadata, RoPE/scaling configuration, and RMSNorm epsilon needed for canonical replay. The resulting verifier key remains small enough for client-held verification (roughly $\sim 26$ MB for Llama 70B). After precomputation, the verifier deletes the weights. The $r_j$ vectors are verifier-secret; if the prover learns them, it can forge passing checks.

**Specification binding.** The deployment is described by four committed specs: $H_{\text{input}}$, $H_{\text{model}}$, $H_{\text{decode}}$, and $H_{\text{output}}$. These bind preprocessing semantics, model configuration, decode policy, and output policy respectively. Unsupported semantics are not left ambiguous: they must either be replayed exactly or rejected explicitly.

### 4.2 Phase 1: Commitment

The provider runs inference normally. The serving path is unchanged except for a tracing layer that captures intermediates alongside execution. In batched serving (e.g., vLLM with paged attention), multiple requests share GPU resources; the tracing layer captures per-request intermediates, extracting per-request activations from the batched computation without modifying the batched kernels.

For every token at every layer (both prefill and decode), the provider records:

- The INT8 input to each of the 7 weight matrices
- The INT32 accumulator output ($W_{\text{i8}} \times x_{\text{i8}}$, exact)
- The requantized INT8 values at each bridge
- The post-attention INT8 output
- The per-tensor quantization scales at each requantization bridge
- The captured pre-final-norm residual needed to anchor the exact final-token tail

After generation completes, the provider builds two Merkle trees [5]. The trees are separate because they serve different access patterns: shell verification (Step 2) opens a few positions from $R_T$, while KV provenance (Step 3) opens the full prefix from $R_{\text{KV}}$. A single tree would force opening all intermediates at every prefix position just to extract the KV values.

- $R_T$ (trace tree): over all intermediates at all tokens. The provider cannot change any activation after committing.
- $R_{\text{KV}}$ (KV tree): over per-token $K, V$ state across all layers. The provider cannot retroactively rewrite earlier tokens' context. This tree allows efficient opening of the full prefix KV without opening the complete trace at every position.

A deployment commitment binds everything outside the forward pass:

$$M = H\big(H_{\text{input}} \parallel H_{\text{model}} \parallel H_{\text{decode}} \parallel H_{\text{output}}\big) \tag{1}$$

The provider also samples a fresh per-request transcript seed $s$, commits $H(s)$ in the receipt, and derives per-token randomness deterministically from $s$ and the token index.

The provider returns the response plus a compact receipt $(R_T, R_{\text{KV}}, M, H(s), N, ...)$. Most responses are never audited. The receipt is the only overhead on the normal serving path.

### 4.3 Phase 2: Verification

The client decides to audit a response. The provider does not know which responses will be challenged or which tokens within them will be opened.

#### *4.3.1 Step 1: Challenge*

The verifier selects $c$ random token positions from the $N$-token response and, for each, chooses $\ell$ layers to open. Opening multiple layers on the same token is preferred — this enables cross-layer consistency checks (Step 4).

#### *4.3.2 Step 2: Shell Verification*

For each challenged token $t$ at each opened layer $i$, the provider opens the INT8 inputs $x_{\text{i8}}$, INT32 accumulators $z_{\text{i32}}$, per-tensor quantization scales, the residual stream values at the layer boundaries, and the post-attention output attn_out_i8 — all with Merkle proofs against $R_T$. The verifier performs four checks:

1. **Freivalds on each shell weight matrix.** For each of the 7 shell matrices, the verifier checks $v_j^{(i)} \cdot x \equiv r_j^T \cdot z \pmod p$. Each check is two dot products in $\mathbb{F}_p$ — $O(n)$. If the prover used wrong weights, false-accept probability is $\leq 1/p$ per matrix. The bound follows from the Schwartz–Zippel lemma: a nonzero linear form over $\mathbb{F}_p$ vanishes on at most a $1/p$ fraction of inputs.
2. **Requantization bridges (exact).** The verifier recomputes the i32 $\rightarrow$ i8 conversion elementwise from the opened accumulators and quantization scales. This is deterministic integer arithmetic. Without this check, the prover could pass Freivalds (correct $r \cdot z$) but feed fabricated i8 values into the next stage.
3. **SiLU (exact).** INT8 inputs take 256 possible values. The verifier checks every element against a precomputed 256-entry lookup table.
4. **RoPE and RMSNorm.** RoPE is recomputed from the position index — deterministic integer-scaled arithmetic. RMSNorm is canonically recomputed from the opened residual stream values; the result is verified in the quantized output space (the recomputed value must requantize to the same i8 as the committed value).

After this step, the verifier has trusted $Q_t$, $K_t$, $V_t$ (as requantized INT8 values with verified RoPE) for the challenged token.

#### *4.3.3 Step 3: KV Provenance*

Attention at token $t$ requires the full prefix $K_{1..t}$, $V_{1..t}$. Shell verification gives trusted values for the challenged token but not the prefix.

For each opened layer $i$, the provider opens the $K$, $V$ values at all prefix positions $j < t$ with Merkle proofs against $R_{\text{KV}}$. The verifier then:

1. Verifies the Merkle proofs — confirming these are the values the prover committed in Phase 1.
2. Randomly samples $k$ earlier token positions $j_1, ..., j_k < t$.
3. For each sampled position $j_s$, runs full shell verification at layer $i$ (Freivalds on all 7 matrices, requantization, RoPE, RMSNorm, SiLU), producing independently verified $K_{j_s}^{(i)}$, $V_{j_s}^{(i)}$.
4. Compares these shell-verified values against the corresponding entries opened from $R_{\text{KV}}$.

If the prover tampered with $f$ out of $n$ prefix positions and the verifier samples $k$:

$$P(\text{catch}) = 1 - (1 - f/n)^k \tag{2}$$

Commitment binding is exact (hash collision resistance). Correctness of unsampled positions is statistical — the prover risks the verifier sampling any corrupted position.

#### *4.3.4 Step 4: Cross-Layer Consistency*

There is no separate proof object here; the consistency check comes from opening multiple layers and verifying the commit-

ted bridges between them. When the verifier opens layers $L$ and $L+1$ on the same token, fake attention at layer $L$ must produce a post-attention output that, after requantization and $W_o$, feeds into layer $L+1$'s RMSNorm consistently with the committed trace. Both sides of this boundary are shell-verified, so the adversary cannot fabricate a consistent bridge without matching the committed values exactly. The more layers opened, the tighter the constraint. In full-audit mode (all layers opened), fake attention at every layer must stay mutually consistent through the entire residual stream.

#### *4.3.5 Step 5: Attention Replay*

This step combines the trusted $Q_t$ from Step 2 with the commitment-verified prefix $K_{1..t}$, $V_{1..t}$ from Step 3. The verifier recomputes attention independently in FP64. Since the KV cache stores post-RoPE values, the verifier dequantizes directly and computes:

1. Scores: $Q_t \cdot K_j/\sqrt{d}$ for all prefix positions $j$
2. Softmax over scores
3. Weighted sum: $\sum \alpha_j \times V_j$

The verifier quantizes the result to INT8 using the quantization scales opened from the trace (committed in $R_T$) and compares element-wise against the committed post-attention output. A response passes if at least a fraction $\tau$ of INT8 elements agree; $\tau$ must be calibrated empirically by measuring the FP16↔FP64 requantization corridor on real model activations (see Section 6).

This pins the attention output to a specific computation over the committed values. Without it, the adversary could fabricate a plausible-looking but wrong attention output — a local-window approximation, suppressed attention to specific positions, or a cheaper model's attention pattern — as long as it satisfied cross-layer consistency.

**Limitations.** The replay proves consistency with the *committed* prefix, not necessarily with true execution at every earlier token. Prefix KV values are commitment-verified (they match $R_{\text{KV}}$) but only statistically anchored to real computation via the sampled shell checks in Step 3. The replay also cannot match the GPU's FP16 attention exactly — the verifier's FP64 reference will differ slightly near quantization bucket boundaries.

#### *4.3.6 Step 6: Final-Token Replay*

The verifier now enters the exact final-token tail. The provider opens the captured pre-final-norm residual for the challenged token, together with the committed decode/output policy and the revealed transcript seed. The verifier then:

1. Applies the final RMSNorm exactly using the committed model spec.
2. Quantizes into the LM-head input space canonically.
3. Checks the LM head with Freivalds and computes logits exactly.
4. Replays the canonical decode policy using the committed decode spec and the per-token seed derived from the revealed transcript seed.
5. Verifies the chosen token and the output-policy behavior (EOS handling, stop strings, max/min stopping rules, and claimed text cleanup) exactly, or rejects the feature explicitly if the deployment claims an unsupported policy.

This step is what makes the final-token boundary exact. The verifier must not derive the token from a hidden state reconstructed through many layers of approximate attention replay and then call the result exact.

### 4.4 Summary

Table 4 summarizes the verification method applied to each operation in the forward pass.

| **Operation** | **Verification** |
|---|---|
| Input / deployment specs | Committed four-spec manifest |
| Input embedding | Table lookup (exact) |
| $W_q, W_k, W_v$ (INT8) | Freivalds |
| Requantize i32 → i8 | Exact recomputation |
| RoPE on $Q$, $K$ | Exact recomputation |
| Attention | Replay + cross-layer |
| $W_o$ (INT8) | Freivalds |
| RMSNorm | Canonical recomputation |
| $W_{\text{gate}}$, $W_{\text{up}}$ (INT8) | Freivalds |
| SiLU $\odot$ up | 256-entry LUT + exact |
| $W_{\text{down}}$ (INT8) | Freivalds |
| Final boundary | Captured pre-final-norm residual |
| LM head | Freivalds + exact logits replay |
| Decode policy | Canonical replay or fail-closed rejection |
| Output policy | Exact replay or fail-closed rejection |

Table 4: Per-layer verification methods

### 4.5 Audit Walkthrough: One Token, Two Layers

Audit token $t = 47$ at layers 12 and 13 of a 70B model, sampling $k = 8$ prefix positions.

1. **Challenge.** Verifier sends $(t = 47, \{12, 13\})$.
2. **Shell.** Provider opens trace at both layers from $R_T$. Verifier runs Freivalds on all 7 matrices per layer (28 dot products total), recomputes all bridges. Yields trusted $Q_{47}$, $K_{47}$, $V_{47}$ at both layers.
3. **KV provenance.** Provider opens $K_{1..46}$, $V_{1..46}$ at both layers from $R_{\text{KV}}$. Verifier picks 8 random earlier positions, runs full shell verification at each, confirms committed $K$, $V$ match.
4. **Cross-layer.** Post-attention output at layer 12 feeds into layer 13′s RMSNorm — both sides shell-verified, so boundary must match exactly.
5. **Replay.** At each opened layer, the verifier recomputes attention in FP64 from shell-verified $Q_{47}$ and the commitment-verified prefix. Quantizes to INT8, compares against committed `attn_out_i8`. Passes if at least fraction $\tau$ of INT8 elements agree at both layers.
6. **Final token.** Provider opens the captured pre-final-norm residual and revealed transcript seed. Verifier applies final RMSNorm exactly, checks the LM head with Freivalds, recomputes logits exactly, replays the canonical decode policy, and confirms the chosen token and output policy.

### 4.6 Data Lifecycle

Table 5 shows what data exists at each stage of the protocol.

| Receipt fields (compact) | Provider retains (ring buffer) | Revealed on audit |
|---|---|---|
| $R_T$ (trace Merkle root) | INT8 inputs to all 7 matrices | Merkle openings from $R_T$ |
| $R_{\mathrm{KV}}$ (KV Merkle root) | INT32 accumulator outputs | Merkle openings from $R_{\mathrm{KV}}$ |
| Spec commitment $M$ | Requantized INT8 at each bridge | Opened intermediates at challenged positions |
| Seed commitment $H(s)$ | `attn_out_i8` + quantization scales | Full prefix $K$, $V$ at opened layers |
| Token count $N$ | Captured pre-final-norm residual | Opened final boundary state + revealed seed |
| | Per-tensor quantization scales | |

Table 5: Data lifecycle: receipt (every response), retained state (RAM ring buffer, discarded after audit window), and audit openings (revealed only when challenged).

# 5 Security Analysis

## 5.1 Security Game

We formalize model-identity soundness via an interactive game between a challenger $\mathcal{C}$ (verifier) and an adversary $\mathcal{A}$ (provider).

**Game**$^{\mathrm{id}}_{\mathcal{A}}(\lambda)$:

1. **Setup.** $\mathcal{C}$ publishes weights $W$ and Merkle root $R_W$. $\mathcal{C}$ samples verifier-secret Freivalds vectors for every shell matrix family and for the LM head, precomputes the corresponding verifier-side checks, and stores the secret verifier key sk. $\mathcal{A}$ receives $W$, $R_W$ but never observes sk.
2. **Commit.** $\mathcal{A}$ receives a prompt, runs inference (using any strategy), and returns a response plus receipt $(R_T, R_{\mathrm{KV}}, M, H(s), N, ...)$.
3. **Challenge.** $\mathcal{C}$ selects $c$ token positions and $\ell$ layers per token uniformly at random. $\mathcal{A}$ learns the challenge only now.
4. **Open.** $\mathcal{A}$ opens the trace at challenged positions with Merkle proofs against $R_T$.
5. **Verify.** $\mathcal{C}$ runs Freivalds and bridge checks at every opened position. Outputs accept or reject.

**Definition (Model-identity soundness).** $\mathrm{Adv}^{\mathrm{id}}_{\mathcal{A}} = \Pr\big[\text{Verify accepts} \wedge \mathcal{A} \text{ used } W'_j \neq W_j \text{ at any Freivalds-checked matrix}\big]$. The protocol is $\varepsilon$-sound if $\mathrm{Adv}^{\mathrm{id}}_{\mathcal{A}} \leq \varepsilon$ for all adversaries $\mathcal{A}$.

## 5.2 Model Swap or Downgrade

**Proposition 1 (Model identity).** *For each Freivalds-checked matrix, if the provider used $W'^{(i)}_j \neq W^{(i)}_j$, the Freivalds check rejects with probability $\geq 1 - 1/p$.*

*Proof sketch.* The adversary commits $(x, z)$ via $R_T$ before learning $r_j$. If $z \neq W^{(i)}_j x$ (wrong weights), let $d = W^{(i)}_j x - z \neq 0$. The check $v^{(i)}_j \cdot x \equiv r^T_j \cdot z \pmod p$ reduces to $r^T_j \cdot d \equiv 0 \pmod p$. Since $d \neq 0$ is fixed before $r_j$ is revealed, and $r_j$ is uniform over $\mathbb{F}^m_p$, exactly $p^{m-1}$ of $p^m$ vectors satisfy this linear equation. Thus $\Pr[\text{accept}] = 1/p \leq 2^{-32}$. □

A single audit of a single token suffices. The guarantee is unconditional — it does not depend on statistical sampling or threshold calibration.

## 5.3 Attention Manipulation

**Proposition 2 (Attention replay bound).** *If attention replay with threshold $\tau$ accepts, the committed post-attention output agrees with the verifier's FP64 recomputation on at least fraction $\tau$ of INT8 elements. Disagreement is bounded by the FP16↔FP64 requantization corridor and the KV sampling boundary.*

The shell locks the inputs ($Q$, $K$, $V$) and outputs (post-$W_o$) of every attention block. Manipulation that alters the INT8 output beyond the corridor — local-window approximations, suppressed context, substituted patterns — produces element-wise disagreement exceeding $\tau$ and fails the replay. The remaining adversarial freedom is bounded by the corridor width and the KV sampling rate.

## 5.4 Fabricated Prefix Context

**Proposition 3 (KV tampering detection).** *If the provider tampered with $f$ out of $n$ prefix positions and the verifier samples $k$, detection probability is $P(\text{catch}) = 1 - (1 - f/n)^k$.*

Committed KV values cannot change after $R_{\mathrm{KV}}$ is sent (hash collision resistance). The verifier anchors the commitment to real computation by sampling earlier positions and running full shell verification. Detection probability increases monotonically with the tampering fraction $f/n$.

## 5.5 Deployment Configuration Tampering

The deployment commitment $M$ binds four specs into the receipt: preprocessing, model configuration, decode policy, and output policy. This includes the tokenizer, chat template, BOS/EOS preprocessing policy, truncation semantics, weight identity, quantization scheme, RoPE/scaling configuration, RMSNorm epsilon, adapter identity, sampler version, temperature/top-k/top-p, penalties, grammar constraints, stop policy, and detokenization semantics. The verifier checks these against known-good values for the intended deployment. Any change to the deployment configuration produces a different committed spec surface.

## 5.6 Assumptions Outside Protocol Scope

The protocol does not assume honest prover hardware or honest provider runtime behavior. Those are the target of verification. The explicit assumptions outside protocol scope are:

1. standard cryptographic assumptions for the hash functions and finite-field checks
2. secrecy of verifier-only material such as the Freivalds vectors
3. no side-channel leakage of verifier-secret material
4. correct execution of the verifier itself

# 6 The Attention Gap

The shell is exactly verifiable because its operations are deterministic or canonically recomputable: INT8 matmuls are checked cryptographically via Freivalds, while requantization, RoPE, SiLU, and RMSNorm are verified by exact or canonical recomputation. Attention ($QK^T$, softmax, $\alpha V$) is computed in FP16/BF16, which is not bit-reproducible across hardware.

The protocol constrains the attention interior from both sides: inputs $(Q, K, V)$ and outputs (post-$W_o$) are exactly verified by the shell. Attention replay directly checks consistency. Cross-layer constraints force fake attention to survive the residual stream.

The remaining adversarial freedom comes from two sources:

1. **FP16↔FP64 replay mismatch.** The verifier's FP64 reference and the GPU's FP16 computation produce slightly different results. The **requantization corridor** — the fraction of INT8 elements that disagree — bounds the adversary's room. This is an open empirical question. High agreement ($> 99\%$) means tight constraint; lower agreement means more room.
2. **Statistical KV anchoring.** The prefix $K, V$ values used in attention replay are commitment-verified (they match $R_{\mathrm{KV}}$) but only statistically anchored to real computation via sampled shell checks. Unsampled prefix positions are not independently verified, so the replay proves consistency with the *committed* prefix, not necessarily with the true execution at every earlier token.

## 7 Provider Costs

### 7.1 Storage

Only the non-derivable retained state requires storage: post-attention INT8 outputs, the associated per-tensor quantization scales, and the captured pre-final-norm residual that anchors the exact final-token tail (everything else is re-derivable):

This storage accounting is only for retained per-token trace state. The deployment commitment $M$ is computed and bound into the receipt at commit time; it is configuration metadata, not non-derivable trace state.

| Model | Layers | hidden_dim | Per token |
|---|---|---|---|
| Llama 8B | 32 | 4096 | $\sim$ 144 KB |
| Llama 70B | 80 | 8192 | $\sim$ 672 KB |
| Llama 405B | 126 | 16384 | $\sim$ 2.1 MB |

Table 6: Per-token storage for non-derivable intermediates

### 7.2 Audit Window

With a short audit window (1–2 minutes), traces fit in a RAM ring buffer with no disk I/O. For Llama 70B on 4× H100, write rate is $\sim$ 1.3 GB/s, requiring $\sim$ 315 GB ($\sim$ 16% of 2 TB system RAM) for a 2-minute window. Longer windows spill to NVMe or networked storage.

Short audit windows require automated auditing — the client's audit decision must be programmatic.

## 8 Related Work

The most direct alternative to VeriLM is proving inference in zero knowledge. Earlier verifiable-inference systems such as SafetyNets [1] and later zk approaches such as zkCNN [2] show that inference correctness can be proved by reducing neural networks to arithmetic-circuit-style verification. However, current ZK systems still impose large prover overheads. For a model that costs dollars per response to serve, this translates to hundreds or thousands of dollars per proved response — prohibitive for production inference.

Related cryptographic ML systems such as Delphi [6] address a different problem: privacy-preserving neural inference, where neither party should reveal its inputs or model. VeriLM instead targets model-provenance auditing for open weights, where the model is public and the goal is to verify that the claimed weights were actually used.

VeriLM makes the opposite tradeoff: verification is interactive (the provider must respond to challenges) and non-transferable (only the verifier who holds the key can check). In exchange, the normal serving path is unchanged and the per-response overhead is a compact receipt. The expensive part — verification — only occurs on the small fraction of responses that are audited.

Trusted execution environments (TEEs) and remote attestation provide another alternative [3], [7]. TEEs can attest that specific code ran on specific hardware, avoiding the overhead of proof systems, but they introduce hardware trust assumptions, limit deployment flexibility, and tie verification to a specific vendor's attestation chain. VeriLM requires no hardware trust beyond standard computation.

| Approach | Provider overhead | Guarantee | Trust assumption | Transf. |
|---|---|---|---|---|
| ZK proofs | 100–1000 × prover | Full computation correct | None (math) | Yes |
| TEEs | Attestation HW | Claimed code ran on attested HW | Hardware vendor | Yes |
| Redundant exec. | 2–3× compute | Outputs agree across replicas | Honest majority | No |
| VeriLM | Compact receipt + rare audits | Claimed weights used (exact); attention consistent (approx.) | Verifier key secrecy + verifier integrity | No |

Table 7: Comparison of verifiable inference approaches.

## 9 Limitations and Extensions

The protocol's practical viability rests on two open empirical questions: the width of the requantization corridor (which bounds the attention gap) and whether storage and bandwidth costs are acceptable to providers at scale. Closing the attention gap entirely would require deterministic attention kernels or stronger proof systems, both of which violate the sidecar design constraint. The final protocol also assumes an autoregressive decoder-only architecture with the committed capture layout; broader architecture support requires additional schema and replay work but does not change the guarantee taxonomy.

Two extensions could tighten the guarantees:

**Score anchoring.** The prover commits pre-softmax attention scores; the verifier spot-checks $\mathrm{score}[t, j] \stackrel{?}{=} Q_t \cdot K_j$ as exact INT32 inner products. This constrains individual score entries independently of the output-level replay. The limitation is that

the GPU computes softmax on FP16 scores, not the INT32 values the verifier checks, so the check cannot chain exactly to the softmax output.

**Batch Freivalds on prefix.** The verifier checks all prefix positions simultaneously using random linear combinations: pick random scalars $\alpha_1 ... \alpha_n$ and check $v \cdot (\sum \alpha_i x_i) \stackrel{?}{=} r^T \cdot (\sum \alpha_i z_i)$. One check covers all $n$ positions with false-accept probability $\leq 1/2^{32}$. This upgrades KV provenance from statistical sampling to exact weight verification at all positions. Cost: $\sim 4 \times$ audit bandwidth, making it better suited as an optional deep audit mode.

## 10 Conclusion

VeriLM demonstrates that meaningful verification of LLM inference is possible without modifying the serving path or requiring expensive proof systems. The protocol's honest decomposition into exact, statistical, approximate, and fail-closed layers allows clients to understand precisely what is and is not guaranteed. A compact receipt imposes negligible overhead on normal serving; the full audit is CPU-feasible for the client.

The most important next step is empirical measurement of the requantization corridor, which will determine how tightly the protocol constrains attention manipulation in practice.

## References


[1] Z. Ghodsi, T. Gu, and S. Garg, "SafetyNets: Verifiable Execution of Deep Neural Networks on an Untrusted Cloud," in *Advances in Neural Information Processing Systems 30*, 2017. [Online]. Available: https://papers.nips.cc/paper/7053-safetynets-verifiable-execution-of-deep-neural-networks-on-an-untrusted-cloud

[2] T. Liu, X. Xie, and Y. Zhang, "zkCNN: Zero Knowledge Proofs for Convolutional Neural Network Predictions and Accuracy," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, 2021, pp. 2968–2985. doi: 10.1145/3460120.3485379.

[3] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, "Remote ATtestation procedureS (RATS) Architecture," technical report RFC9334, Jan. 2023. doi: 10.17487/RFC9334.

[4] R. Freivalds, "Fast Probabilistic Algorithms," in *Mathematical Foundations of Computer Science 1979*, in Lecture Notes in Computer Science, vol. 74. Springer, 1979, pp. 57–69. doi: 10.1007/3-540-09526-8_5.

[5] R. C. Merkle, "A Digital Signature Based on a Conventional Encryption Function," in *Advances in Cryptology – CRYPTO '87*, in Lecture Notes in Computer Science, vol. 293. Springer, 1987, pp. 369–378. doi: 10.1007/3-540-48184-2_32.

[6] P. Mishra, R. Lehmkuhl, A. Srinivasan, W. Zheng, and R. A. Popa, "Delphi: A Cryptographic Inference Service for Neural Networks," in *29th USENIX Security Symposium (USENIX Security 20)*, USENIX Association, 2020, pp. 2505–2522. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/mishra

[7] J. Ménétrey *et al.*, "Attestation Mechanisms for Trusted Execution Environments Demystified," in *Distributed Applications and Interoperable Systems*, in Lecture Notes in Computer Science, vol. 13272. Springer, 2022, pp. 95–113. doi: 10.1007/978-3-031-16092-9_7.